Assembly Basics / Addressing Modes

- CMPE310/CMSC313
- Dr. Kidd

Acknowledgements

- These lectures are duplicates of Dr. Kidd's CMPE310 lectures.
- These lectures are largely from Dr. Chintan Patel, UMBC, with significant modifications and additions from myself.

Constants and Other Conventions

Constants

- To specify a hexadecimal constant, proceed it with a “0x”.
  - E.g. 0x1234, 0xAF
- To specify a binary constant, proceed it with a “0b”
  - E.g. 0b10110000, 0b10101011
- To specify a decimal constant, don’t proceed it with anything
  - E.g. 15, 64001, 256
- To specify an octal constant, place a “0” (zero, not the letter ‘O’) in front
  - E.g. 0373, 0177777
  - Yes, this is inconsistent
- Contents of memory address

  mov 0x45AFFFFF,%eax
  - means move the contents at the memory address 0x45afffff into the eax register
- An immediate value

  mov $0x55AFFFFF,%eax
  - means move the constant 0x55AFFFFF into the EAX register.
  - A ‘$’ in front of a value says that the value is not to be used as a memory address

Specifying Size of Data Operated Upon

- Specifying the data size
  - To specify the size of the data operated upon by the instruction use a suffix
- Use ’b’ for 8-bit (byte)
- ‘w’ for 16-bit (word)
- ‘l’ for 32-bit (long), and
- ‘q’ for 64-bit (quad word)
  - E.g. addb memvalue,%al
  - E.g. addw memvalue,%ax
  - E.g. addl memvalue,%eax
  - E.g. addq memvalue,%rax

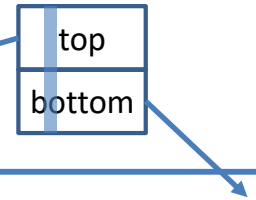
Data Formats

ASCII

- Use hexadecimal. Much easier to see the relationships and remember.

Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym
0	0	NUL	32	20		64	40	@	96	60	`
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	$	68	44	D	100	64	d
5	5	ENQ	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	TAB	41	29	)	73	49	I	105	69	i
10	A	LF	42	2A	*	74	4A	J	106	6A	j
11	B	VT	43	2B	+	75	4B	K	107	6B	k
12	C	FF	44	2C	,	76	4C	L	108	6C	l
13	D	CR	45	2D	-	77	4D	M	109	6D	m
14	E	SO	46	2E	.	78	4E	N	110	6E	n
15	F	SI	47	2F	/	79	4F	O	111	6F	o
Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	

ASCII

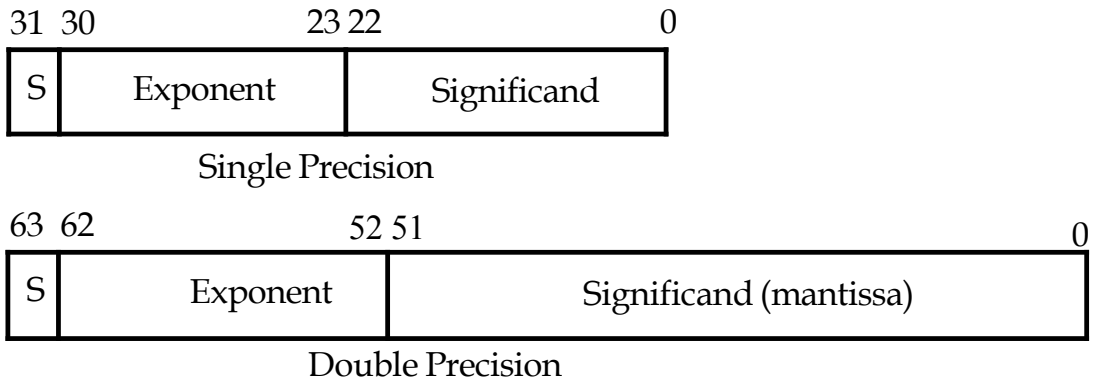


Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym
0	0	NUL	32	20		64	40	@	96	60	`
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	$	68	44	D	100	64	d
5	5	ENQ	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	TAB	41	29	)	73	49	I	105	69	i
10	A	LF	42	2A	*	74	4A	J	106	6A	j
11	B	VT	43	2B	+	75	4B	K	107	6B	k
12	C	FF	44	2C	,	76	4C	L	108	6C	l
13	D	CR	45	2D	-	77	4D	M	109	6D	m
14	E	SO	46	2E	.	78	4E	N	110	6E	n
15	F	SI	47	2F	/	79	4F	O	111	6F	o

Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym	Dec	Hex	Sym
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	

Table 1: ASCII (American Standard Code for Information Interchange)

Floating Point Formats



- For single precision, the sign bit + 8-bit exponent + 24-bit mantissa = 33 bits !
  - The mantissa has a hidden 1 bit in the leftmost position that allows it to be stored as a 23-bit value.
  - The mantissa is first normalized to be >= 1 and < 2, e.g., 12 in binary is 1100, normalized is 1.1 X 23.
  - The exponent is also biased by adding 127 (single) or 1023 (double), e.g. the 3 in the previous example is stored as 127 + 3 = 130 (0x82).

Floating Point Formats and Directives

Dec	Bin	Normal	Sign	Expon	Mantissa	
+12	1100	1.1×2^3	0	10000010 1000000	00000000	00000000

- There are two exceptions:
  - The number 0.0 is stored as all zeros.
  - The number infinity is stored as all ones in the exponent and all zeros in the mantissa. (The sign bit is used to indicate + or - infinity.)

- Directive is ***.float*** for single precision, and ***.double*** for double precision:
  - **.float** 1.2
  - **.double** 1.e+10
  - **.double** 3.14159265358979323846 2

Format of an assembly instruction:

LABEL	OPCODE	OPERANDS	COMMENT
DATA1:	.byte	0b00001000	#Define DATA1 as decimal 8
_start:	mov	%ebx,%eax	#Copy ebx to eax

LABEL:

Stores a symbolic name for the memory location that it represents.

OPCODE:

The instruction (or directive) itself.

OPERANDS:

- A register, an immediate or a memory address holding the values on which the operation is performed.
- There can be from 0 to 3 operands.

Assembly Directives

- Do not generate code/data themselves
- Describes
  - How to interpret data (e.g. .byte and .ascii)
  - Configuration information (e.g. .globl and .section)
  - Symbols / macros (e.g. .equ)
  - Control (e.g. .include and .org)
  - Etc

Assembly Directives

- ASCII: Stored using an assembler directive ***.ascii***, ***.asciz***:
- Word-sized (***.word***) and doubleword-sized data (***.long***):
  neg_exponent: .long -100
- Little endian: Least significant byte is always stored in the lowest memory location.

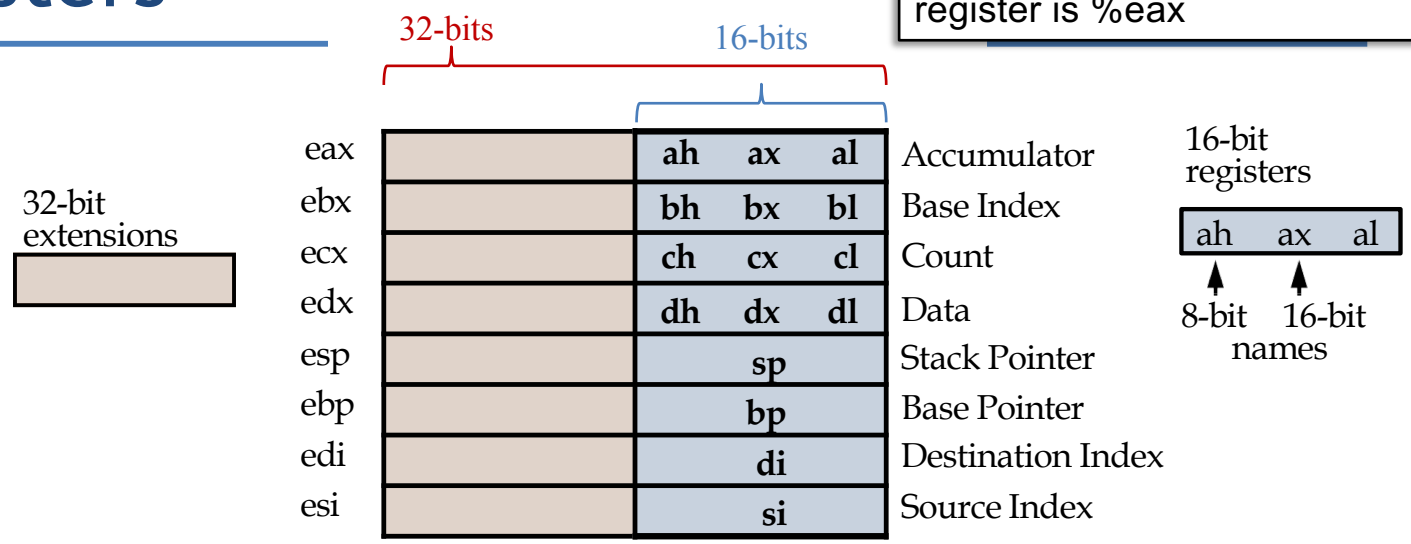
```
floatstr:
            .asciz ”Float number -> %f “
            .byte 10,0        #linux eol
mainl_str:
            .asciz "Rectangular Areas”
            .byte  10,0       #linux eol
temp_buf:
            .rept 200         #repeat 200 times
            .byte 0
            .endr

.set temp_buf_size, .-temp_buf #set temp_buf size
```

0x303	*Storage of the number 0x1234*	
0x302	0x12	←High-order byte
0x301	0x34	←Low-order byte

Data Addressing Modes

Data: Registers

NOTE: in GAS, a register is proceeded by '%' so the eax register is %eax



- Let's cover the data addressing modes using the ***mov*** instruction.
  - Data movement instructions move data (bytes, words and doublewords) between registers and between registers and memory.
  - Only the ***movs*** (strings) instruction can have both operands in memory. (DMA inst.)
  - Most data transfer instructions do not change the **EFLAGS** register.

Data: Immediate

- To operate on a value, use '$'
  - movw $1,%eax #move the const '1' into EAX register
  - CMPL $9,%RBP #compare contents of RBP register with value '9'
- Encodes the constant directly into the instruction
  - Called an 'immediate' value
  - Fast instruction since doesn't need separate memory access

Data: From Memory

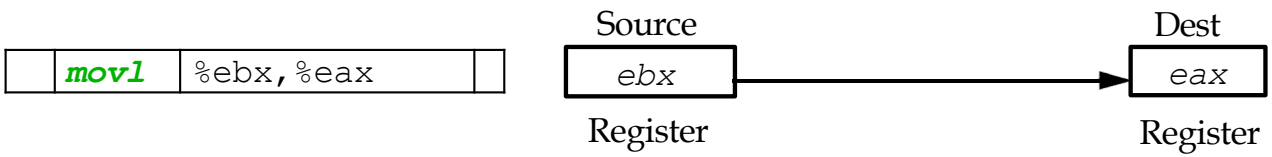
- To operate on a value from memory, use memory address
  - movw data,%eax #move contents at address data to EAX reg
  - CMPL maxnum,%RBP #compare contents at addr maxnum with RBP reg
- Address of data in instruction
  - Operates on contents at address (i.e. pointed to)
  - Slower than immediate since requires memory operation
  - Much more flexible than immediate

Addressing: Where to Find Data

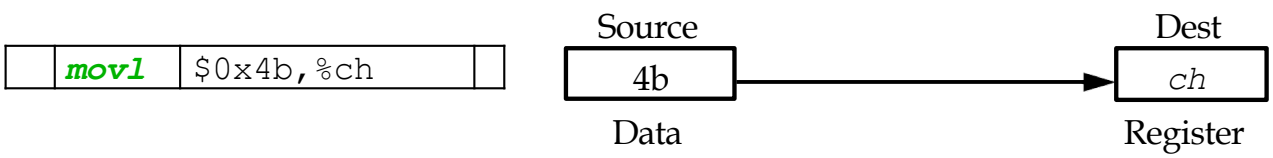
- Register: In a register
- Immediate: In the instruction
- Direct: In the specified address
- Indirect: The contents specifies where to find the data
  - Register Indirect: (%ebp) – The data is pointed to by the contents of EBP reg
  - Base relative: Indirect but displaced by amount
    - -4(%ebp) means 4 locations down from address in EBP
  - Indexed: Like array addressing
    - (%ebp+%esi) means EBP holds start (e.g. of array), ESI holds index into (e.g. index of array)

Register, Immediate, Direct, Displacement

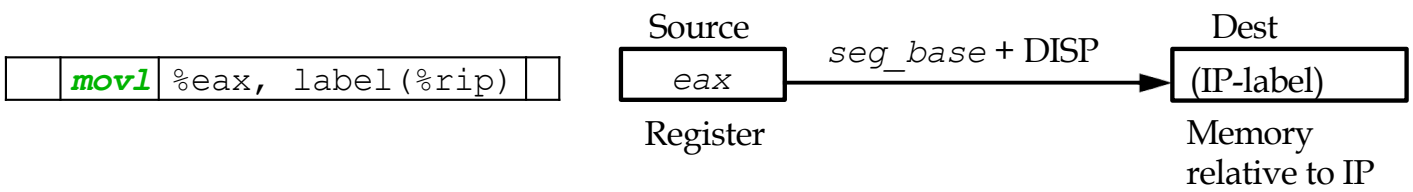
- Register



- Immediate



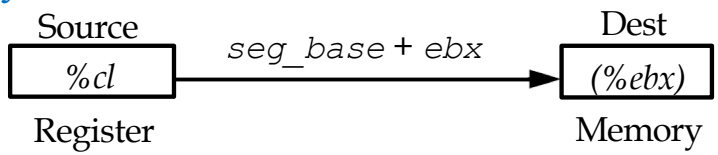
- Direct (eax), Displacement (other regs)



Register indirect, Base-index, Register relative

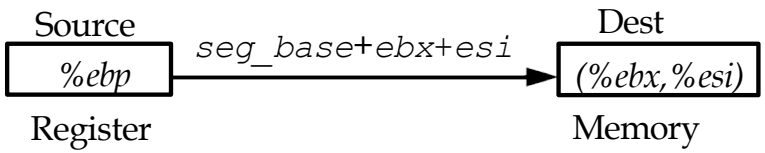
- Register Indirect
  - Any of *eax*, *ebx*, *ecx*, *edx*, *ebp*, *edi* or *esi* may be used.

```
movw  %cl,(%ebx)
```



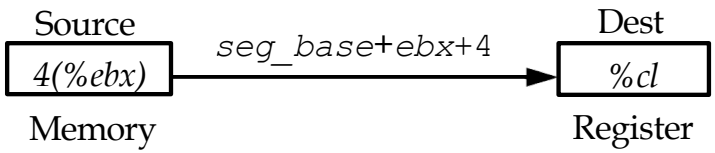
- Base-plus-index
  - Any combination of *eax*, *ebx*, *ecx*, *edx*, *ebp*, *edi* or *esi*.

```
movl  %ebp,(%ebx,%esi)
```



- Register relative

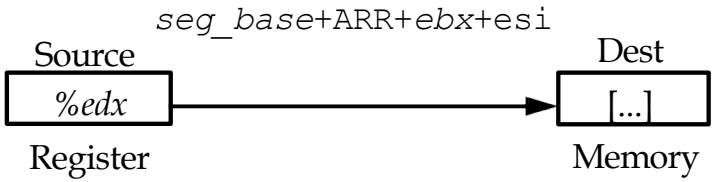
```
movl  4(%ebx),%cl
```



Base relative-index, Scaled-index

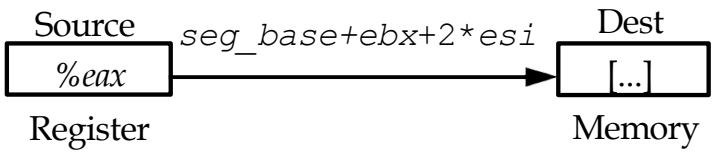
- Base relative-plus-index

```
movl %edx,ARR(%ebx,%esi)
```



- Scaled-index

```
movl %eax,(%ebx,%esi,2)
```



  - Scaling factors can be 2×, 4× or 8×.

NOTATION: Data Length

- According to the GAS specification
  - Must always specify a length on an instruction
  - ”mov %eax, %ebx” isn’t legal
  - “movl %eax,%ebx” is legal
- Actually, most implementations
  - Accept “mov %eax, %ebx”
    - Implies length of “mov” as “movl” since %eax is 16-bits
  - As long as instruction length is not ambiguous

Register addressing, Immediate addressing

- Register addressing
  - Note: ***mov*** really COPIES data from the source to destination register.
  - Never mix an 16-bit register with a 32-bit, etc. For example
    - mov %bx,%eax #ERROR: NOT permitted.
  - None of the ***mov*** instruction effect the **EFLAGS** register.

- Immediate addressing:
  - The value of the operand is given as a constant in the instruction stream.
    - mov $0x12345,%eax
  - Use '0b' for binary, '0' for octal and nothing for decimal.
    - $0123 is the octal value 123
      - Note that $097 produces an error as '9' doesn't exist in octal
  - ASCII data requires a set of apostrophes:
    - mov $'A',%eax #Moves ASCII value 0x41 into **eax**.

Register addressing, Immediate addressing Examples

- Register and immediate addressing example:

	.globl	_start	
	.text		#start of the code segment.
_start:			
	mov	$0,%eax	#Immediate addressing
	mov	$0x0000,%ebx	
	mov	$0,%ecx	
	mov	%eax,%esi	#Register addressing.
	...		

- Direct addressing:
  - Transfers between memory and al, ax and eax.
  - Usually encoded in 3 bytes, sometime 4:

	mov	DATA1,%al	#Copies a byte from DATA1.
	mov	0x4321,%al	#Some assemblers don't allow this.
	mov	0x1234(%ds),%al	
	mov	%ax,DATA2	#Copies a word to DATA2.

Displacement Example

- Displacement Example:

mov	*DATA1,%cl*	#Copies a byte from DATA1
mov	SUM,%edi	#Copies a doubleword from SUM

  - Displacement instructions are encoded with up to 7 bytes (32-bit register and a 32-bit displacement).
- Direct and displacement addressing example:

				.globl _start
0000				.data
0000	10		DATA1:	***.byte 0x10***
0001	00		DATA2:	***.byte 0***
0000			_start:	.text
0017	A0	0000 R		***mov DATA1,%al***
001A	8B	1E 0001 R		***mov DATA2,%bx***

  - Note: Direct addressing (using **al**) requires 3 bytes to encode while Displacement (using **bx**) requires 4.

Register Indirect addressing

- Offset stored in a register is added to the segment register.

	mov	*(%ebx),%ecx*	
	mov	*(%ebx),(%edi)*	

- The memory to memory ***mov*** is allowed with string instructions.
  - Any register EXCEPT **esp** for the 80386 and up.
  - For **eax**, **ebx**, **ecx**, **edx**, **edi** and **esi**: The data segment is the default.
  - For **ebp**: The stack segment is the default.
- Some versions of register indirect require instructions that explicitly specific the data size, e.g. ***movb*** and ***movw***

	mov	*(%edi),%al*	#Clearly a byte-sized move
	mov	*$0x10,(%edi)*	#Ambiguous, assembler can't size

- Does [**edi**] address a byte, a word or a double-word?
  - Use:

	movb	*$0x10,(%edi)*	#A byte transfer

Base-Plus-Index addressing

- Effective address computed as:
  - seg_base + base + index.

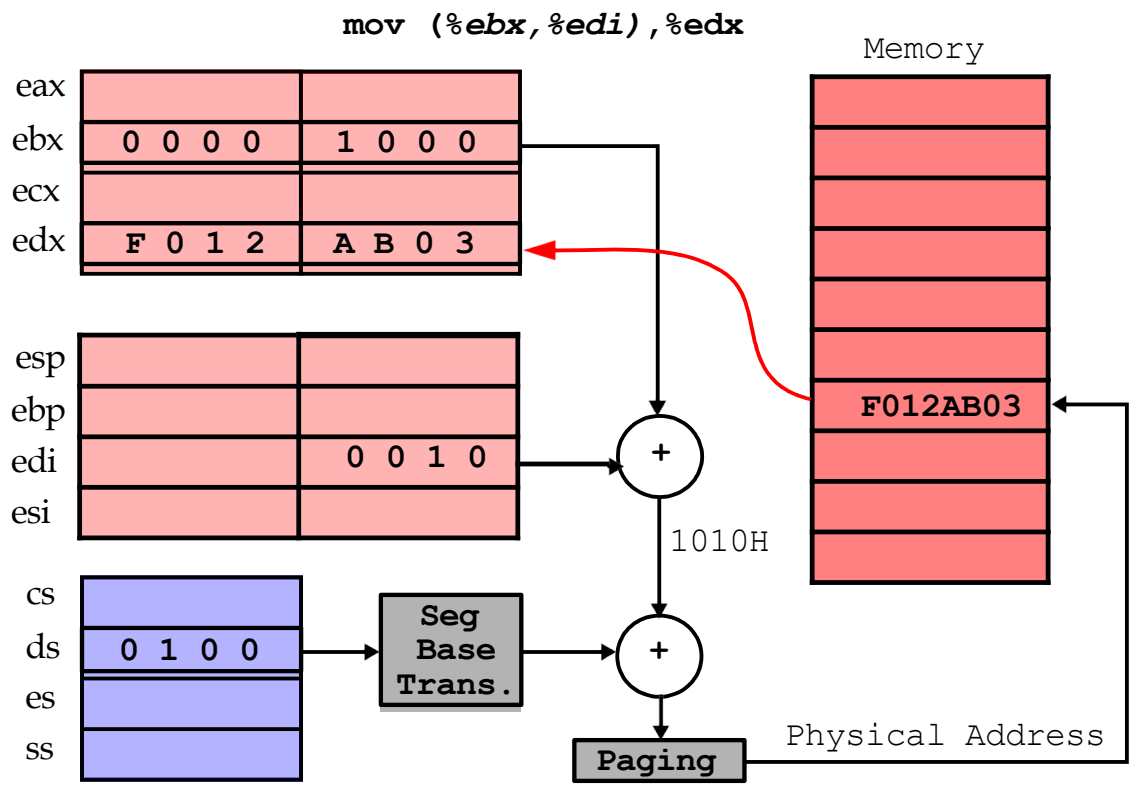
- Base registers: Holds starting location of an array.
  - **ebp** (stack)
  - **ebx** (data)
  - Any 32-bit register except **esp**.

- Index registers: Holds offset location.
  - **edi**
  - **esi**

	mov	`(%ebx,%edi),%ecx`	`#Data segment copy`
	mov	`(%ebp,%esi),%ch`	`#Stack segment copy`
	mov	`(%eax,%ebx),%dl`	*#EAX* as base, *EBX* as index

  - Any 32-bit register except **esp**.

Base-Plus-Index addressing



Register-relative, Base-relative Index Examples

- Register Relative addressing:
  - Effective address computed as: seg_base + base + constant.

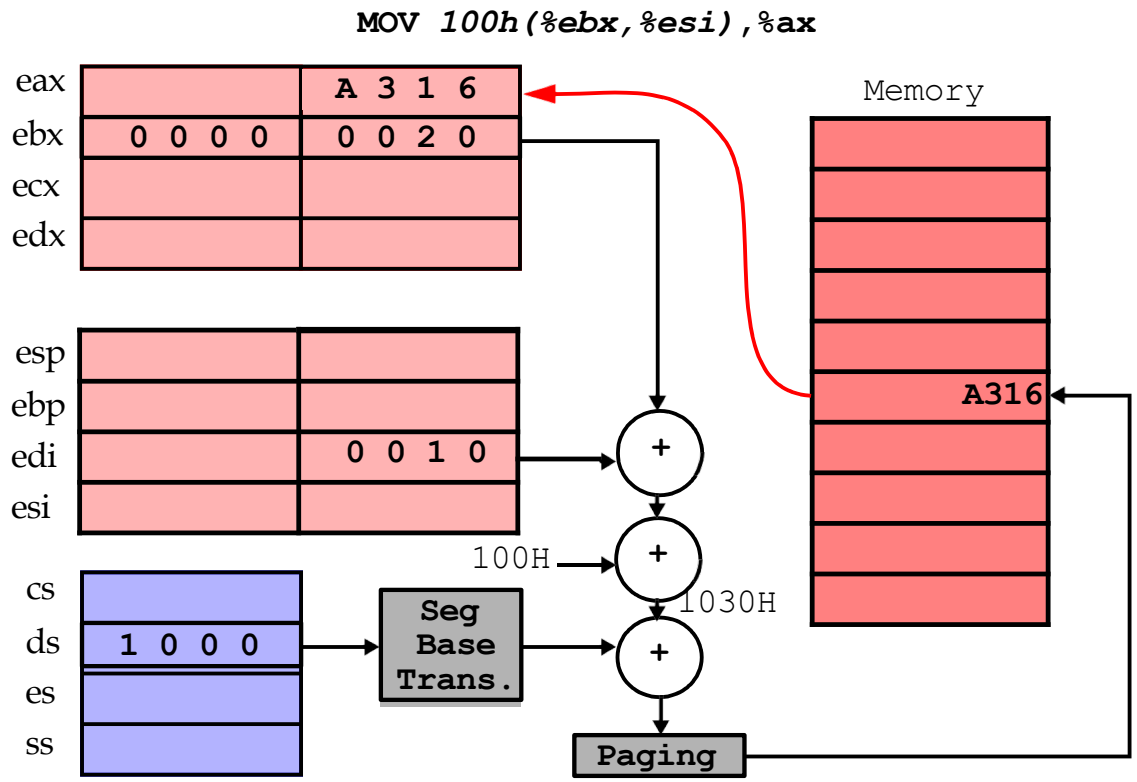
	mov	1000h(%*ebx*),%eax	#Data segment copy
	mov	%bl,ARRAY(%*esi*),*BL*	#Constant is ARRAY
	mov	(LIST+2)(%*esi*)	#Both LIST and 2 are constants
	mov	(LIST-2)(%*esi*),%edx	#Subtraction

  - Same default segment rules apply with respect to **ebp**, **ebx**, **edi** and **esi**.
    - Displacement constant is any 32-bit signed value.
- Base Relative-Plus-Index addressing:
  - Effective address computed as: seg_base + base + index + constant.

	mov	*20h(%ebx,%edi)*,%dh	#Data segment copy
	mov	FILE(%*ebx,%edi*),%ax	#Constant is FILE
	mov	%dh,(LIST+4)(%*ebp,%esi*),dh	#Stack segment copy
	mov	(FILE+2)(%ebx,%*ecx*),%eax	#32-bit transfer

  - Designed to be used as a mechanism to address a two-dimensional array.

Base Relative-Plus-Index addressing



Code, Data and Stack Addressing

Data/Code Addressing Modes

- Scaled-Index addressing:
  - Effective address computed as: seg_base + base + constant*index.

	mov	(%ebx,%ecx,4),%eax	#Data segment DWORD copy
	mov	%cx,-100h(%eax,%edi,2)	#Whow!
	mov	%eax,ARRAY(%ecx,4),%eax	#Stack segment copy

  - Code Memory-Addressing Modes:
  - Used in jmp and call instructions.
  - Three forms:
    - Direct
    - PC-Relative
    - Indirect

- Direct
  - Absolute jump address is stored in the instruction following the opcode.

Code Addressing Modes

- Intersegment jump:
  - This far jmp instruction loads **cs** with 1000H and **eip** with 00000000H.
    - A far call instruction is similar.

Opcode	Offset(low)	Offset(high)	Segment(low)	Segment(high)
EA	0000	0000	00	10

- PC-Relative
  - A displacement is added to the **eip** register.
  - This constant is encoded into the instruction itself, as above.

- Intra-segment jumps:
  - Short jumps use a 1-byte signed displacement.
  - Near jumps use a 4-byte signed displacement.
  - The assembler usually computes the displacement and selects the appropriate form.

Code Addressing Modes

- Indirect
  - Jump location is specified by a register.
  - There are three forms:
    - Register
      - Any register can be used: **eax**, **ebx**, **ecx**, **edx**, **esp**, **ebp**, **edi** or **esi**.

	jmp	`*%eax`	`#eax contains cs addr`

    - Register Indirect
      - *Intra*segment jumps can also be stored in the data segment.

	jmp	`*(%ebx)`	`#Jump address obtained from ds`

    - Register Relative

	jmp	`*TABLE(%ebx)`	`#Jump TABLE which ever seg defined`
	jmp	`*2(%edi)`	`#edi+2 obtained from ds`

      - The '*' isn't necessary but makes the indirection visually explicit.

Stack Addressing Modes

- The stack is used to hold temporary variables and stores return addresses for procedures.
  - ***push*** and ***pop*** instructions are used to manipulate it.
  - ***call*** and ***ret*** also refer to the stack implicitly.

- Two registers maintain the stack, **esp** and **ss**.
  - A **LIFO** (Last-in, First-out) policy is used.
  - The stack grows toward lower address.
  - Data may be pushed from any of the registers or segment registers.
    - Data may be popped into any register except cs.

	popf		`#Pop from stack to EFLAG`
	pushf		`#Pushes EFLAG register`
	push	*$0x1234*	`#Pushes 0x1234`
	pushl	*(%ebx)*	`#Pushes double word in data seg`
	pushal		`#eax,ecx,edx,ebx,esp,ebp,esi,edi`
	pop	*%eax*	`#Pops 4 bytes`